I highly recommended that when you're troubleshooting with print statements, you use one of the techniques I discuss in Chapter 16, *Running Tests Faster and Running Faster Tests*, on page 319, to run just one test at a time for clarity. Fast tests help a lot here, because the quicker you can get data on the screen, the faster you can iterate, and using print statements for test troubleshooting is definitely a rapid-iteration process.

Here are some ways to use print statements in troubleshooting.

You Are Here

Print little "You're here" messages as a cheap alternative to program traces. (Ruby does have a program tracer, called `tracer`, but it's going to be wild overkill for most debugging purposes.) I usually go with a simple `p "here"` and, if I need a second one, `p "there"`. It's a fast way to determine if the method you're looking at is even called during the test invocation.

For the most part I use these as one-and-done quick checks to see what's going on. It's not uncommon for a check like this to save me significant time banging my head against the code looking at a method that isn't even being called.

Also helpful here is the Putsinator gem,[2] which annotates every `p` and `puts` with the line number that generated them. This is helpful in tracking down that one line of output in the test suite that you can't otherwise trace. Also, using Rubocop will by default catch stray `p` statements and flag them before you commit them to production.

You May Ask Yourself, "How Did I Get Here?"

At any point in Ruby, you can determine the name of the method that called the method you're in by using the cryptic incantation `caller_locations(1,1)[0].label`. `caller_locations` is a kernel method that returns the call stack as an array of objects. You can also use the `caller` method, which returns the call stack in a different format. The two arguments are index and length, with index `0` being the current method, and index `1` being the immediate caller. So we have an array of length `1`, returning the first element via `[0]` and then taking that object's `label` attributes, which is the method name. You're encouraged to tweak the call a bit to see what other data is around: see ruby-doc.org for official docs.[3]

I recommend you keep this one on speed dial in the form of a helper method in your `test_helper` or `rails_helper` file.

2. https://github.com/tobytripp/Putsinator
3. http://www.ruby-doc.org/core-2.1.1/Kernel.html#method-i-caller_locations